PROJECT PRESENTATION STUDY GUIDE

Fire Suppression Tower

Computer vision, flame localization, target tracking, training results, safety boundaries, and weekly updates

Prepared from the repository state on October 23, 2026

The central idea: the laptop detects and localizes a flame, then computes simulated aiming corrections. The ESP32, physical motors, safety interlocks, and pump are separate hardware responsibilities and are not yet integrated.

1. Presentation-ready overview

30-SECOND SUMMARY

"The Fire Suppression Tower uses a camera and a dedicated one-class YOLO model to detect and localize flames. The software selects the strongest detected flame, verifies it across three consecutive frames, calculates its pixel error from the camera center, and converts that error into bounded pan-and-tilt corrections. Motor commands are currently simulated. ESP32 communication, physical motor control, IR confirmation, person lockout, and pump activation are not yet integrated."

The system pipeline

```
Logitech camera
↓
OpenCV captures a frame
↓
YOLO produces flame boxes and confidence scores
↓
Select the strongest flame and calculate its center
↓
Require three consecutive detections
↓
Compare flame center with camera center
↓
Apply dead zone, proportional gain, and command limit
↓
Print a simulated PAN/TILT command
↓
Display detections, tracking state, error, and output
```

Laptop responsibilities

- Camera capture
- Neural-network inference
- Flame localization
- Tracking calculations
- Simulated commands

ESP32 responsibilities

- IR sensor input
- Physical motor drivers
- Mechanical limits
- Pump output
- Real-time safety behavior

2. Flame detection and localization

Object detection versus classification

A classifier would only report that an image contains a flame. YOLO performs **object detection**, so it also returns where each flame appears.

Output	Meaning	How the project uses it
Class	The type of object	The dedicated model has one class: `0 = flame`.
Confidence	Strength of the model's detection	Reject weak predictions and rank multiple flames.
Bounding box	Rectangle around an object	Draw the flame and calculate its center.
Center point	Midpoint of the bounding box	Target input for pan-and-tilt tracking.

Important thresholds

> **Confidence threshold: 0.30.** Predictions below 30% confidence are rejected. Lowering it can find weaker or more distant flames but can also increase false positives.
>
> **IoU threshold: 0.35.** Intersection over Union measures box overlap. During non-maximum suppression, it helps remove redundant boxes around the same flame.

Confidence is not a perfectly calibrated real-world probability. It is most useful for filtering and ranking predictions.

Multiple flames

The detector retains and displays all flame detections, but the highest-confidence flame supplies the target coordinates. This is simple and deterministic, but the selected target could switch between flames from frame to frame.

Why the model is one-class

A specialized flame-only model narrows the problem and dataset. Person detection exists only as separate experimental code; it is not integrated into the dedicated detector and must not be presented as an active safety lockout.

3. Confirmation and tracking states

A flame must appear in three consecutive frames before the tracker issues a nonzero correction. This reduces movement caused by a single noisy prediction, reflection, or bright object.

State	Meaning	Command behavior
`NO_TARGET`	No valid flame or invalid frame dimensions	Pan and tilt are zero; confirmation resets.
`CONFIRMING`	Flame detected, but fewer than three consecutive frames	Pan and tilt remain zero.
`TRACKING`	Confirmed flame is outside the center dead zone	Bounded correction is produced.
`CENTERED`	Confirmed flame is inside both dead zones	Pan and tilt are zero.
`DISABLED`	Tracking is turned off in configuration	Pan and tilt are zero.

> **Known limitation:** confirmation checks that a flame exists in consecutive frames. It does not establish a persistent object identity or prove that every detection belongs to the same physical flame.

4. Controller math

Step 1: calculate image error

```
error_x = target_x - frame_width / 2
error_y = target_y - frame_height / 2
```

- Negative X means left; positive X means right.
- Negative Y means above; positive Y means below.
- Direction-inversion settings can reverse either physical axis later.

Step 2: proportional correction

```
pan correction  = Kp_pan  × error_x
tilt correction = Kp_tilt × error_y
```

The current proportional gains are `0.05` for both axes. This is a **P controller**, not a full PID controller. It has no integral term for accumulated error and no derivative term for damping.

Step 3: dead zone and clamping

- The dead zone is ±40 pixels on each axis.
- Inside the dead zone, that axis receives a zero correction.
- Each output is clamped to the range −25 through +25.

The dead zone reduces jitter near the center. Clamping prevents a large image error from producing an unbounded request.

> **EXAMPLE TO MEMORIZE**
>
> For a 1920×1080 frame, the center is `(960, 540)`. If the flame center is `(1200, 460)`, the error is `(+240, -80)`.
>
> ```
> Pan  = 0.05 × 240  = +12
> Tilt = 0.05 × -80  = -4
>
> Simulated message: PAN:12.00,TILT:-4.00
> ```

5. Motor boundary and firmware status

The laptop-side motor module formats a future newline-delimited serial message such as:

```
PAN:12.50,TILT:-7.00
```

`SIMULATION_MODE` is currently `True`, so commands are printed rather than transmitted. If simulation is disabled, the program intentionally raises an error because actual serial output has not been implemented.

The detector attempts to issue `PAN:0.00,TILT:0.00` during shutdown or an exception. This is a best-effort software stop; it is not a substitute for a hardware emergency stop or communication watchdog.

> **Firmware reality:** the ESP32 currently initializes serial communication and prints a boot message. Its loop still contains TODOs for IR input, motor aiming, and pump control. The physical suppression system is not operational from this code.

6. Model training and data

Training process

- The dedicated model starts from pretrained YOLO weights.
- The dataset is normalized to exactly one class: flame.
- Training requires CUDA device 0 and does not silently fall back to CPU.
- Mixed precision is enabled to improve GPU speed and memory use.
- Early stopping ends training after validation stops improving for the configured patience.
- Ultralytics writes `best.pt` for the best validation checkpoint.

Hard negatives

Hard negatives resemble flames but contain no actual fire. Examples include lamps, reflections, flashlights, red or orange objects, phone screens, and unlit candles. Empty YOLO label files teach the detector that these scenes should produce no flame box.

Data-quality controls

- Positive images can contain individually labeled multiple flames.
- The collector reports overexposure and sharpness information.
- Exposure and focus can be adjusted for small or distant flames.
- Nearly identical frames should not be split across training, validation, and test sets because that would inflate reported performance.

CUDA and TensorRT

The PyTorch checkpoint is stored as `.pt`. It can be exported to a TensorRT FP16 `.engine` for faster NVIDIA GPU inference and reduced memory use. The engine is tied to its TensorRT/GPU environment, and the current engine uses a fixed 640×640 input.

The camera can capture at a higher resolution, such as 1920×1080, while YOLO processes the image at the configured inference size. The resizable display window is independent of both.

7. Results and correct interpretation

The recent `candle_hard_negatives_640` run reached its best strict validation score around epoch 12:

Metric	Value	Interpretation
Precision	≈ 0.906	About 90.6% of predicted boxes were correct under that validation setup.
Recall	≈ 0.910	About 91.0% of labeled flames were detected.
mAP@0.50	≈ 0.944	Strong average precision with a moderate 50% box-overlap requirement.
mAP@0.50:0.95	≈ 0.490	Performance across increasingly strict localization requirements.

> **Do not overclaim:** runs should only be directly compared when they use the same held-out validation data and evaluation settings. The latest experimental run also produced `NaN` validation losses in its final row, which should be investigated before relying on it.

The detector currently prefers `runs/candle_yolo/weights/best.engine`, then `best.pt`. New experimental checkpoints are not automatically promoted to the deployment default.

8. What changed this week

Committed on October 21

- Added a resizable detector display window.
- Separated display dimensions from camera capture and inference resolution.
- Made early-stopping patience configurable in the training monitor.
- Corrected the training monitor's epoch-count display.

Current in-progress working-tree changes

- Added centralized tracking configuration.
- Added the proportional flame-tracking controller.
- Added a simulated motor-command interface.
- Integrated tracking state and corrections into the live detector.
- Added visual overlays for dead zone, target error, state, and controller output.
- Added best-effort zero commands during shutdown.
- Added unit tests for tracking math and message formatting.

> **Presentation wording:** “The committed update improved the detector display and training monitor. Target tracking and simulated motor integration are the current in-progress work.”

9. Verification and limitations

Verified in the current repository

- Six tracking/protocol unit tests pass.
- The active detector, tracker, configuration, and motor-interface modules compile.
- Tests cover direction signs, command clamping, dead zone, confirmation, lost-target reset, axis inversion, disabled mode, and message formatting.

Not proven by those tests

- Live-camera detection accuracy
- Physical motor direction or mechanical calibration
- USB serial communication
- IR confirmation
- Person safety lockout
- Pump behavior
- End-to-end suppression safety

Current engineering limitations

- Commands are abstract corrections, not calibrated motor angles or steps.
- The strongest target can change in a multi-flame scene.
- There is no persistent target identity or motion prediction.
- The controller has not been tuned against real motor dynamics.
- Camera exposure, reflections, and small distant flames can affect performance.
- There is no hardware watchdog, limit-switch handling, or emergency stop in this software.

10. Likely questions and concise answers

Why use YOLO?

> It provides real-time object locations, not only a yes-or-no classification, so the same output can support aiming.

Why use three-frame confirmation?

> It reduces reactions to isolated false detections while adding only a small delay.

Why use a dead zone?

> Bounding-box centers fluctuate slightly. The dead zone prevents constant motor hunting when the target is approximately centered.

Why use proportional control?

> It produces small corrections for small errors and larger corrections for large errors. It is simple, interpretable, and suitable for initial tests.

Why clamp the command?

> It establishes a maximum request and prevents an extreme pixel error from producing an extreme control output.

What happens if the target disappears?

> The confirmation counter resets, the state becomes `NO_TARGET`, and both corrections return to zero.

What happens with multiple flames?

> All are displayed, but the highest-confidence flame is used as the aiming target. Target switching is a known limitation.

Can it move the motors now?

> No. It currently formats and prints simulated commands. ESP32 serial output is intentionally unimplemented.

Can it activate the pump?

> No. Pump control is not implemented.

Does it guarantee nobody is sprayed?

> No. Person detection and other safety interlocks are not integrated or validated. The system must remain in simulation.

Why require CUDA?

> GPU inference and training are much faster, and refusing silent CPU fallback prevents the system from appearing functional while running with unacceptable latency.

Does a 90% confidence mean a 90% real-world probability?

> Not necessarily. Detection confidence is useful for filtering and ranking, but it is not automatically a calibrated probability.

11. Recommended next steps

1. Create a fixed held-out test set and compare candidate models on identical data.
2. Investigate the `NaN` validation loss in the latest experiment.
3. Select and explicitly configure the intended deployment checkpoint.
4. Implement ESP32 serial messages with acknowledgments, sequence handling, and a timeout.
5. Calibrate pixel error to physical motor direction, speed, and angle.
6. Add limit switches, a hardware emergency stop, and a communication watchdog.
7. Integrate and validate person detection and IR confirmation.
8. Implement a safety state machine before connecting the pump.
9. Run controlled end-to-end tests with suppression output simulated.
10. Only then consider controlled physical pump testing.

> **Bottom line:** this is currently a computer-vision and simulated-targeting prototype, not a deployable autonomous fire-suppression system.

12. Final review checklist

Before presenting, make sure you can explain these without reading:

- The complete camera-to-command pipeline
- Object detection versus classification
- Confidence versus IoU
- The purpose of three-frame confirmation
- The proportional-control equation
- The 1920×1080 numerical example
- Why the dead zone and clamp exist
- What simulation mode does
- What is committed versus still in progress
- What the six passing tests prove—and what they do not prove
- The current safety limitations
- The next integration steps

Repository references: `computer-vision/fire-detection/detect_candle_flame.py`, `target_tracking.py`, `config.py`, `motor_interface.py`, `test_target_tracking.py`, and `firmware/src/main.cpp`.